

Histogram Equalization in PyOpenCL and NumPy

Habib Kirolos

September 11, 2026

Contents

1	Introduction	2
2	Histogram Equalization	2
2.1	Problem Statement	2
2.2	General Procedure, and Parallel Patterns Therein	2
3	Implementation	4
3.1	Technologies Used	4
3.1.1	PyOpenCL	4
3.1.2	NumPy	4
3.2	PyOpenCL Implementation	4
3.2.1	Color Space Conversion	5
3.2.2	Histogram	6
3.2.3	Addition Scan	7
3.3	NumPy Implementation	9
4	Results and Benchmarks	11
4.1	Benchmarks Done	11
4.2	Benchmark Results	11
4.3	Conclusions	11

Contents

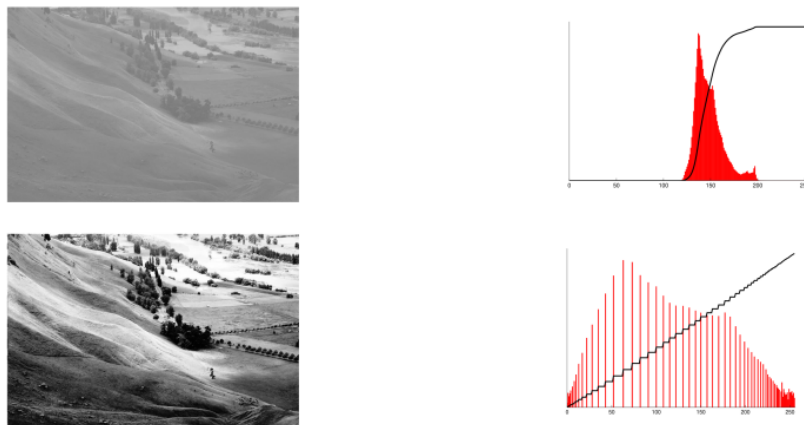
1 Introduction

For this project we have implemented histogram as a case study in GPU programming for parallelizing tasks, the task in this case being a fundamental image processing procedure known as histogram equalization.

We here present the problem, discuss the technical decision taken throughout its implementation, show our implementation, and its performance with respect to a cpu computed baseline implemented in NumPy

2 Histogram Equalization

2.1 Problem Statement



Histogram equalization consists in altering the pixel values of an image point-wise in a way aimed at spreading out their values as much as possible, this rather informal requirement is formalized as changing the values in such a way that the cumulated histogram of the image values is as linear as possible, as shown in the image above.

2.2 General Procedure, and Parallel Patterns Therein

To compute the histogram equalization of an image one needs to first compute the histogram of the value they want to equalize, this histogram may be computed for only one channel of the image, or for all channels of the image, this first step already presents itself as an established parallel programming problem, that being the computation of histograms.

From this histogram a cumulative histogram is then computed by performing an addition scan on the histogram, as the equalization formula requires the cumulative histogram.

This formula, relating the original value to the equalized value, is later computed for all pixel values in the image, resulting in the equalized image.

$$round(\frac{CDF(hist)(hist - 1)}{pixels})$$

This transformation is computed pointwise and its values for all pixels are computed independantly of each other, meaning it therefore falls under the map parallel programming pattern.

As an optional preprocessing (and postprocessing) step, an input RGB image may be converted to a more workable color space, such as HSL or HSV, before processing, and converted back to RGB after processing, this may, for instance, be done to only equalize the L channel of an HSL image, which is what our implmenetation is going to do.

Being computed in parallel and independantly for every pixel, the RGB to and from HSL conversion of pixels also falls under the map parallel programming pattern.

3 Implementation

3.1 Technologies Used

3.1.1 PyOpenCL

The de facto standard technology used for GPU programming (and for this exam) is CUDA, the author has sadly found many issues making proper usage of CUDA not advisable for this project, including, but not limited to, unfitness of web development environments, and lack of hardware support for the machine under test.

The project has been therefore implemented using the `OpenCL` API, as it was the closest in API and performance characteristics to CUDA. The project had initially been developed using the `PyOpenCL` library, with plans to later port it to C and have it directly use `libOpenCL`, this plan was later scrapped and the `PyOpenCL` version was deemed fit as a final artefact.

Despite the choice of `OpenCL`, over `CUDA`, the remainder of this report shall still use `CUDA` terminology (thread, thread block, grid, ...) over the corresponding `OpenCL` terminology (work item, work group, ndrange, ...).

Due to a software update breaking `OpenCL` support on the author's machine the project has been also developed using `docker` to maintain an environment with working drivers for the author's hardware.

3.1.2 NumPy

Out of fear that python runtime overhead could introduce a bias in performance measurements, the planned C CPU baseline implementation has instead been developed using `NumPy`, so that any runtime overhead between the GPU and CPU versions of the program would have been equal for both sides.

3.2 PyOpenCL Implementation

The `PyOpenCL` version of the code has been laid out according to the following pipeline

- Host image is read from file, host histogram is initialized to all zeros
- Device image and histogram are initialized and the host image and histogram values are copied therein (initialized as `OpenCL` buffers with the `COPY_HOST_PTR` flag)
- Device image is converted from RGB to HSL inplace (one kernel launch)

- Histogram of L channel in device image computed into device histogram (one kernel launch)
- Addition scan of histogram is computed inplace to produce cumulative histogram (heirarchical scan, 3 kernel launches)
- Device image L channel normalized inplace (one kernel launch)
- Device image converted from HSL back into RGB (one kernel launch)

3.2.1 Color Space Conversion

Color space conversions are, as previously stated, computed pointwise and independantly for each pixel in the image, every thread acts on one pixel of the image.

Every thread of the grid acts on only one pixel of the image, and a 1d grid of threads is spawned, as no 2d behaviour is required for a pointwise transformation of the image, every thread accesses global memory 3 times to read the pixels it must transform, and 3 times to write the transformed pixels back into the image. Given that no repeated reads are required and that accesses are already quite local no corner turning was performed.

The full **OpenCL** C code for the conversion from rgb to hsl is here provided for illustrative purposes

```
#define MAX2(a, b) ((a>b)?a:b)
#define MIN2(a, b) ((a<b)?a:b)
#define MAX3(a, b, c) ((a>b)?MAX2(a,c):MAX2(b,c))
#define MIN3(a, b, c) ((a<b)?MIN2(a,c):MIN2(b,c))

uchar f2uchar(const float mag) {
    return (uchar)fmin(255, (mag * 256.0f));
}
float uchar2f(const uchar mag) {
    return ((float)mag) / 255.0f;
}

uchar3 rgb2hsl_pixel(const uchar r, const uchar g, const uchar b) {
    const uchar u_max = MAX3(r,g,b);
    const uchar u_min = MIN3(r,g,b);

    // r, g, and b as floats I can operate on
    const float rf = uchar2f(r);
```

```

    const float gf = uchar2f(g);
    const float bf = uchar2f(b);
    const float f_max = MAX3(rf, gf, bf);
    const float f_min = MIN3(rf, gf, bf);

    const float lf = (f_max+f_min)/2.0f;
    if(u_max == u_min)
        return (uchar3){0, 0, f2uchar(lf)};
    else {
        const float df = f_max - f_min;
        const float sf = (lf>0.5f) ?df/(2.0f-f_max-f_min) :df/(f_max+f_min);
        float hf;
        if(u_max == r) hf = (gf-bf) / df+(g<b?6.0f:0.0f);
        if(u_max == g) hf = (bf-rf) / df+2.0f;
        if(u_max == b) hf = (bf-rf) / df+4.0f;
        hf /= 6.0f;
        return (uchar3){f2uchar(hf), f2uchar(sf), f2uchar(lf)};
    }
}

__kernel void rgb2hsl(const __global uchar* rgb_img,
                    __global uchar* hsl_img,
                    const uint n_pixels) {
    const uint i = get_global_id(0);
    if(i<n_pixels) {
        const __global uchar*rgb_pxl = rgb_img + (i*3);
        __global uchar*hsl_pxl = hsl_img + (i*3);
        const uchar3 hsl = rgb2hsl_pixel(rgb_pxl[0],
                                         rgb_pxl[1],
                                         rgb_pxl[2]);

        hsl_pxl[0]=hsl.x;
        hsl_pxl[1]=hsl.y;
        hsl_pxl[2]=hsl.z;
    }
}

```

3.2.2 Histogram

Histogram computation also treats the image as a 1d grid of pixels so launched grid was once more 1d, to avoid contention of accessing shared global memory thread blocks all hold a private partial histogram in local memory, and

only flush the partial histogram to global memory after a barrier ensuring all values in the area allotted to the block have been counted in the local histogram.

Contention to same locations, both for the local and the global histograms, are handled through atomic operations.

The code for a full histogram is presented below for illustrative purposes, though please note the kernel invoked in our image processing pipeline only acts on one channel of a 3 (or 4) channel image, with a strided access pattern to global memory.

```
__kernel void full_hist(const __global uchar* input,
                        __global uint* global_hist,
                        __local uint* local_hist,
                        const uint input_size,
                        const uint hist_size){
    const uint gi = get_global_id(0);
    const uint li = get_local_id(0);
    if(li < hist_size) local_hist[li] = 0;
    barrier(CLK_GLOBAL_MEM_FENCE | CLK_LOCAL_MEM_FENCE);
    if(gi < input_size)
        atomic_add(&local_hist[input[gi]], 1);
    barrier(CLK_GLOBAL_MEM_FENCE | CLK_LOCAL_MEM_FENCE);
    if(li < hist_size)
        atomic_add(&global_hist[li], local_hist[li]);
}
```

3.2.3 Addition Scan

The cumulative histogram is performed using a 2 level heirarchical addition scan, the scan for both levels has been computed using a brent kung adder schema, though given the size of the histogram (our code assumes 8 bit depth, meaning histogram is going to be at most 256 elements) this choice was quite arbitrary.

OpenCL lacks any mechanism to have a barrier act on an entire grid, to achieve such synchronization we've had to drop barriers between the 3 steps of a heirarchical scan in favour of implementing the 3 phases as 3 different kernels, and enqueueing them in order in the host code (unless explicitly specified OpenCL runs all kernels in a queue in the order in which they were added, asynchroneous order execution must be explicitly requested)

The brent kung code is quite illegible so for illustrative purposes the below code is for a heirarchical scan using the kogge stone adder schema

instead (both schemas have been implemented then brent kung was chosen for no particular reason).

```
__kernel void ks_block_scan(__global uint* global_data,
                            const uint global_data_size,
                            __local uint* local_data,
                            const uint local_data_size) {
    const uint global_idx = get_global_id(0);
    const uint local_idx = get_local_id(0);
    const bool is_active = (global_idx < global_data_size);

    local_data[local_idx]=is_active?global_data[global_idx]:0;
    barrier(CLK_LOCAL_MEM_FENCE);

    uint tmp;
    for(uint stride = 1; stride < local_data_size; stride*=2) {
        if (local_idx >= stride)
            tmp = local_data[local_idx - stride];
        barrier(CLK_LOCAL_MEM_FENCE);
        if (local_idx >= stride)
            local_data[local_idx] += tmp;
        barrier(CLK_LOCAL_MEM_FENCE);
    }

    if(is_active) global_data[global_idx] = local_data[local_idx];
}

// expected to be launched with only one work group
// this kernel, together with the above and below kernels,
// provides *one* level of heirarchical scan
__kernel void ks_last_elt_scan(__global uint* global_data,
                               const uint global_data_size,
                               __local uint* chunk_sum_scan,
                               const uint number_of_chunks,
                               const uint single_chunk_size) {
    const uint local_idx = get_local_id(0);
    const uint chunk_sum_global_idx =
        (single_chunk_size * local_idx) +
        (single_chunk_size - 1);
    const bool is_active = (chunk_sum_global_idx < global_data_size);
```

```

    chunk_sum_scan[local_idx]=is_active
        ?global_data[chunk_sum_global_idx]
        :0;
    barrier(CLK_LOCAL_MEM_FENCE);

    uint tmp;
    for(uint stride = 1; stride < number_of_chunks; stride*=2) {
        if (local_idx >= stride)
            tmp = chunk_sum_scan[local_idx - stride];
        barrier(CLK_LOCAL_MEM_FENCE);
        if (local_idx >= stride)
            chunk_sum_scan[local_idx] += tmp;
        barrier(CLK_LOCAL_MEM_FENCE);
    }

    if(is_active)
        global_data[chunk_sum_global_idx] = chunk_sum_scan[local_idx];
}

__kernel void filling_pass(__global uint* global_data,
                           const uint global_data_size,
                           const uint single_chunk_size) {
    const uint global_idx = get_global_id(0);
    uint prev_chunk_end_idx = global_idx - (global_idx % single_chunk_size);
    prev_chunk_end_idx -= (prev_chunk_end_idx != 0);
    const uint curr_chunk_end_idx = prev_chunk_end_idx + single_chunk_size;
    const uint prev_chunk_end = global_data[prev_chunk_end_idx];
    const bool is_active = (global_idx < global_data_size);

    barrier(CLK_GLOBAL_MEM_FENCE);
    if(is_active
        && prev_chunk_end_idx
        && (global_idx != curr_chunk_end_idx))
        global_data[global_idx] += prev_chunk_end;
}

```

3.3 NumPy Implementation

The NumPy version was developed with the goal of being as idiomatic NumPy as possible, at the possible cost of performance, this was done to better compare how you'd usually do this in NumPy with how you'd usually

do this in OpenCL.

The NumPy implementation was laid out according to the following pipeline

- Image is split into R, G, and B channels (transposed first then split into 3 cells of transposed array)
- R, G, and B channels are "undiscretized" from `np.uint8` into `np.float`
- continuous R, G, and B channels are used to compute continuous H, S, and L channels (not inplace)
- L channel undergoes histogram normalization (requires discretization as an intermediate step)
- H, S, and normalized L channel are turned back into R, G, and B channels
- the normalized R, G, and B channels are then discretized back into `np.uint8` and used to construct the normalized image (requires another transposition)

As hinted to above this is a different set of operations from the PyOpenCL code, but porting the degree of index fiddling done in OpenCL to NumPy proved problematic and not too idiomatic, leading to the two transpositions instead. Given how both transpositions are $O(n)$ this adds a lot of overhead to the NumPy version, as we shall see in the benchmarks.

4 Results and Benchmarks

4.1 Benchmarks Done

5 images were chosen hoping to cover as wide a range of possible inputs as possible, and the two implementations were compared in time needed to equalize these images, to "denoise" the time measurements all images were normalized 10 times, and the sum of times was taken as the value of interest, the 5 images were.

- a flat red, square image
- an HD image of the Mona Lisa
- a grayscale photograph
- a color photograph of a landscape
- a color photograph of italian journalist Germano Mosconi

4.2 Benchmark Results

	Germano	Red	Mona Lisa	Landscape	Grayscale
Size	421x748	100x100	11146x7479	796x1200	800x528
NumPy	0.659s	0.021s	223.8s	2.016s	0.835s
PyOpenCL	0.023s	0.012s	4.863s	0.041s	0.025s
Speedup	28.64	1.642	46.03	48.13	33.35

4.3 Conclusions

The `OpenCL` version has been found to be significantly faster than the `NumPy` versions on most images, achieving a speedup of around 40 for all but one tested images.

The 1.6 speedup for the flat red image is likely due to a combination of factors such as

- Small size of the image, making the cost of transfer between host and device quite expensive relative to work done on the device side
- Degenerate histogram of the image, leading to high contention while computing image histogram, likely slowing gpu implementation down further